

Understanding and Mitigating the Video-Action Generalization Gap via Temporal Ratio

Anonymous Author(s)

Affiliation

Address

email

Abstract: Generative video foundation models exhibit strong compositional priors, yet world-action models (WAMs) and video-action models (VAMs) often lose these priors after finetuning on robotic action data. We refer to this discrepancy as the **video-action generalization gap**. In this paper, we systematically investigate this gap by evaluating a comprehensive design space of VAMs, demonstrating that standard design choices yield no emergent explanation pattern. To explain this behavior, we introduce the Temporal Ratio (TR), an attention-based measure of how strongly the action head relies on future latent rollouts relative to the anchored current frame. TR has two key properties: first, a model’s structural reliance on future-predictive latents, measured via TR, acts as a predictor of its compositional generalization capacity; second, it natively fluctuates based on task phase, shifting attention to future frames during planning and reverting to the present frame for precise manipulation. Finally, based on these findings, we propose an inference-time adaptive guidance method, which exploits this intrinsic feature attention pattern to dynamically amplify compositional video conditioning signals precisely when the policy relies on future rollouts. Evaluated on the LIBERO benchmark and real-world tasks, our approach mitigates the OOD-ID compositional generalization gap. More details: temporal-ratio.github.io

Keywords: Video-Action Models, Compositional Generalization

1 Introduction

Compositional generalization [1, 2, 3] is essential for autonomous manipulation: a robot should recombine seen objects, receptacles, and primitives into unseen task sequences. Existing policies often fail under such recompositions because they overfit to visual correlations such as object coordinates [4, 5, 6, 3]. For example, a policy trained on “put the cream cheese in the bowl” and “put the bowl on top of the cabinet” may still fail on “put the cream cheese on top of the cabinet,” despite involving only previously seen objects, receptacles, and primitives. An emerging direction is to use generative video foundation models (VFMs) [7, 8, 9] for action learning, leveraging video models trained on internet-scale video data with strong compositional generalization by capturing temporal coherence, object interactions, and physical plausibility.

Consequently, the key challenge is how to transfer these pretrained video priors into robotic control. Current World-Action Models (WAMs) [10, 11] and Video-Action Models (VAMs) [12, 13, 14] typically either (1) predict future video [15, 16] (or corresponding latent features [12, 13]) and infer actions through inverse dynamics, or (2) jointly model video and action tokens within a unified transformer [10, 11, 17, 18, 19, 20]. However, despite strong in-domain performance, these methods consistently fail to preserve the compositional generalization abilities of their underlying video backbones (Fig. 1, Top) after finetuning on action data [21]. We refer to this discrepancy as the **video-action generalization (VAG) gap**. This motivates the following research question: what is the right interface between VFMs and action prediction that preserves compositional priors?

In this work, we examine a class of VAMs that use latent features from pre-trained video foundation models and learn a latent inverse-dynamics action head [12, 13, 14]. Such models preserve the video model’s generative prior by feeding latent video features to an action head trained with flow matching [22]. To understand the properties of this interface, we systematically study its design space across backbone adaptation and video-latent feature extraction choices. This reveals a fragmented manifold of the in-distribution (ID) and out-of-distribution (OOD) performance gap (Fig. 1, Top), motivating a more mechanistic diagnostic.

Our key observation is that compositional generalization depends on whether the action head actually uses the predicted future rollout. We introduce the **Temporal Ratio** (TR), the attention mass assigned by the action head to future latent frames relative to the anchored current frame. Illustrated in Fig. 1 (Bottom): TR is strongly correlated with a model’s underlying generalization capacity. While success rates look like noise, TR acts as a direct predictor of the ID-OOD performance gap. This gives us an opportunity to close the ID-OOD performance gap by treating TR as a runtime signal to implement inference-time guidance. We also observe that TR varies with task phase: it rises during “planning phase” (e.g., what object to pick, where to place) and drops during precise local manipulation (e.g., grasping or placement). We exploit this intrinsic property via *TR-Adaptive Guidance*, which amplifies language and plan conditioning when the policy enters a planning regime and relaxes guidance during precise manipulation. We evaluate on LIBERO compositional tasks [23, 4] and real-world bimanual tasks, improving OOD success while preserving ID performance.

2 Design Space of Video-Action Models

We study a particular class of VAMs similar to the design of [12, 13]. As shown in Fig. 2, the architecture combines (1) video denoising via flow matching with partially denoised video feature extraction and (2) an action head that denoises actions from the video features from (1). We explore this design space to understand how a video backbone facilitates action generation.

2.1 Background

Video foundation backbone. We use a flow-matching video backbone with velocity field v_θ . Given clean video latents x_0 and noise $\varepsilon \sim \mathcal{N}(0, I)$, the noisy latent at video noise level $\sigma_v \in [0, 1]$ is $x_{\sigma_v} = (1 - \sigma_v)x_0 + \sigma_v\varepsilon$. The video model predicts $v_\theta(x_{\sigma_v}, c, \sigma_v) \approx \varepsilon - x_0$ under text condition c :

$$\mathcal{L}_{\text{video}} = \mathbb{E}_{x_0, \varepsilon, \sigma_v} [\|v_\theta(x_{\sigma_v}, c, \sigma_v) - (\varepsilon - x_0)\|^2]. \quad (1)$$

At target video noise level σ_v , during video denoising, we extract features from k -th block of the Cosmos-2.5 2B DiT backbone [7] (fixed at $k = 20$ out of 28 total blocks following [12]): $F_\theta^{(k)}(x_{\sigma_v}, c, \sigma_v) \in \mathbb{R}^{T \times (H \times W) \times D}$, where $t = 0$ is the anchored clean observation (inpainted at every denoising step) and $t > 0$ are predicted future latent frames.

Action head integration. The action head π_ϕ is a transformer policy (300M Gemma Transformer [25, 24, 26]) that consumes flattened video tokens, a projected proprioceptive token, and noisy action tokens as illustrated in Fig. 2. For clean action sequence a_0 , action noise ε_a , and action

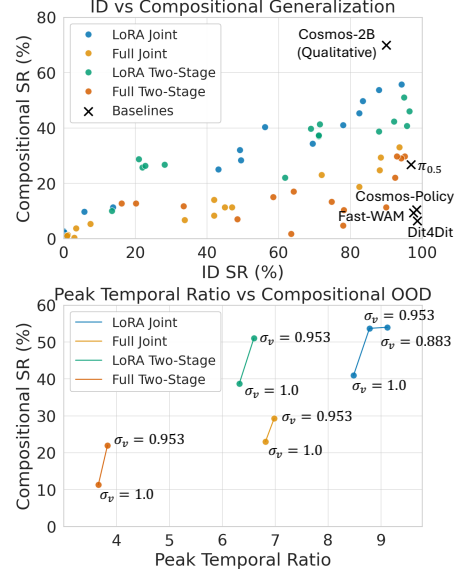


Figure 1: **Interpretable metric to explain performance gap between ID and compositional OOD.** (Top) We study the design space of VAMs over video feature-extraction noise level σ_v and other design choices, finding a fragmented ID-OOD performance gap without a clear consensus. (Bottom) We propose Temporal Ratio (TR) as a diagnostic tool to interpret OOD performance and eventually improve it.

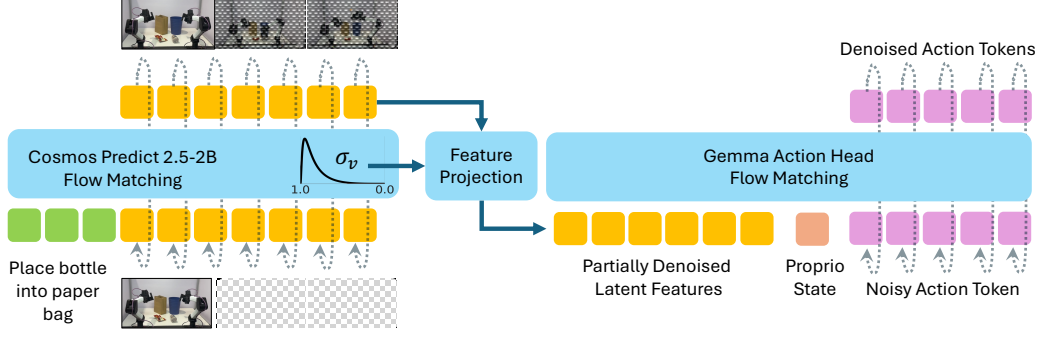


Figure 2: **Latent VAM architecture.** We replace the VLM backbone in π_0 [24] with Cosmos-Predict 2.5 Video DiT [7]. From the current image and language instruction, plausible future rollout is partially denoised; intermediate DiT features and the video noise level are projected into the action head, then concatenated with proprioception and noisy action tokens for action denoising.

87 noise level σ_a , we define $a_{\sigma_a} = (1 - \sigma_a)a_0 + \sigma_a \epsilon_a$ and train

$$\mathcal{L}_{\text{action}} = \mathbb{E}_{a_0, \epsilon_a, \sigma_a} [\|v_\phi(a_{\sigma_a}, F_\theta^{(k)}, s, \sigma_a) - (\epsilon_a - a_0)\|^2]. \quad (2)$$

88 The action head uses bidirectional attention over the video prefix and causal attention over action
89 tokens [24, 26]. We provide training-inference pseudocode in App. A.

90 2.2 Design Axes

91 The design space is defined by two primary groups of axes: training-time configurations governing
92 backbone adaptation, and inference-time parameters determining feature extraction behavior.

93 **Training axes.** (1) *Finetuning strategy*: We compare parameter-efficient adaptation via Low-Rank
94 Adaptation [27] (training 2–6% of parameters at $\eta = 10^{-4}$) against full finetuning of the video DiT
95 backbone ($\eta = 10^{-6}$). (2) *Training mode*: We evaluate *joint training*, which optimizes the weighted
96 objective $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{action}} + \lambda \mathcal{L}_{\text{video}}$, enabling action gradients to directly reshape the DiT feature
97 space. This is contrasted with *two-stage training*, where the backbone is first finetuned on $\mathcal{L}_{\text{video}}$
98 and subsequently frozen during action head optimization on $\mathcal{L}_{\text{action}}$.

99 **Inference axes.** (1) *Video noise level*: We extract features at varying values of $\sigma_v \in [0.0, 1.0]$ along
100 the video denoising flow trajectory. Larger σ_v yields noisier, less defined future latents, whereas
101 smaller σ_v produces highly denoised, semantically structured rollouts. (2) *Prediction horizon*: We
102 vary the temporal extent of the predicted rollout, T .

103 3 Systematic Analysis of the Video-Action Generalization Gap

104 We begin by asking: *when does compositional generalization happen in VAMs?* To answer this, we
105 conduct a wide range of ablation studies over the design axes introduced above: backbone adap-
106 tation, coupled/decoupled video-action training and video feature-extraction strategies. These axes
107 determine how the action head uses a language-conditioned, partially denoised latent rollout with
108 a clean anchor frame ($t = 0$). Hence, we use the design sweep to identify choices that preserve ID
109 control while enabling compositional behavior. Summarized in Fig. 3, we discuss three key findings.

110 **Finetuning strategy changes ID and OOD success differently.** Full finetuning [13] can aggres-
111 sively fit the in-domain tasks by specializing the latent rollout distribution to demonstrated trajec-
112 tories. LoRA [12] restricts the update and tends to preserve more pretrained temporal structure.
113 Crucially, ID and OOD performance do not scale proportionally: *Joint-LoRA* yields the strongest
114 OOD performance, while *Two-Stage LoRA* is more robust when averaged across σ_v and T .

115 **The best video noise level is neither fully noisy nor fully denoised.** At high noise levels ($\sigma_v \rightarrow$
116 1.0), future tokens lack coherent temporal structure, forcing the action head to anchor to the present
117 state [12, 13]. At very low noise levels ($\sigma_v \rightarrow 0.0$), future tokens become highly structured, but the
118 policy is more vulnerable to confident hallucinated trajectories [15]. Performance therefore peaks at
119 an intermediate noise level.

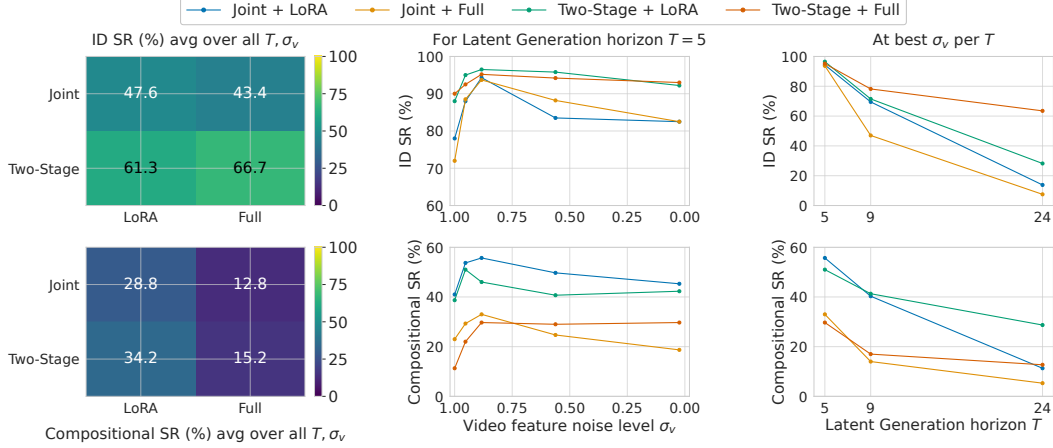


Figure 3: **Systematic exploration of the design space of VAMs.** (Left) We summarize the performance of finetuning strategy and coupled/decoupled video-action training. (Middle) We show the change of ID-OOD SR as the video noise level for feature extraction decreases. Performance peaks at an intermediate σ_v . (Right) We show the change of ID-OOD SR as the temporal horizon of videos increases. While longer temporal horizons better preserve object-goal relationships, this does not necessarily translate into higher SR. For each column, (Top) is ID SR and (Bottom) is OOD SR.

120 **Longer video prediction improves video-level generaliza-**
 121 **tion, but does not necessarily improve action success.** Increasing the prediction horizon exposes more object-goal
 122 structure and can improve the video rollout’s compositional
 123 plan qualitatively. However, longer horizons also increase the
 124 chance of implausible artifacts and do not automatically make
 125 the action head follow the correct plan. As a result, better
 126 video-level generalization does not always translate into higher
 127 task success. More details are provided in App. C.

129 **Motivating example: video-action disagreement.** The frag-
 130 mented ablation landscape motivates a closer look at how the
 131 action head interprets latent video features. As shown in Fig. 4,
 132 the same video rollout can be used or ignored depending on
 133 the task regime. In an ID LIBERO-long task (Top), the video
 134 model may hallucinate a wrong future while the action head
 135 still executes a correct memorized behavior. In a composi-
 136 tional OOD task (Bottom), the video model may propose the
 137 correct plan, but the action head ignores it and follows an ID trajectory. This failure mode cannot be
 138 diagnosed from the video rollout alone: a correct imagined future is only useful if the action policy
 139 follows it. Since video-feature utilization emerges implicitly from underlying design choices rather
 140 than explicit architectural constraints, we require a metric to quantify exactly how much the action
 141 head relies on the imagined future rollout relative to the current-state anchor.

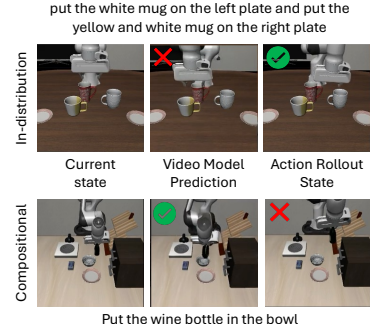


Figure 4: **Video-action disagreement.** (Top) The action head disagrees with an incorrect video prediction and executes correctly. (Bottom) The action head ignores a correct compositional prediction and executes incorrectly.

142 4 Understanding and Mitigating the VAG Gap via Temporal Ratio

143 The design sweep and disagreement examples show that VAM success depends not only on whether
 144 the video backbone predicts a plausible future, but also on whether the action head routes through
 145 that future during action denoising. We therefore introduce the Temporal Ratio (TR), an attention-
 146 based measure of the action head’s reliance on the predicted rollout. At each replan step, action
 147 tokens attend over video tokens containing the current frame and predicted future frames. Let A_ℓ
 148 be the head-averaged attention map from action tokens to video tokens at action-head layer ℓ , after
 149 softmax. Partition video tokens into current-frame tokens $\mathcal{V}_0$ and future-frame tokens $\mathcal{V}_+$.

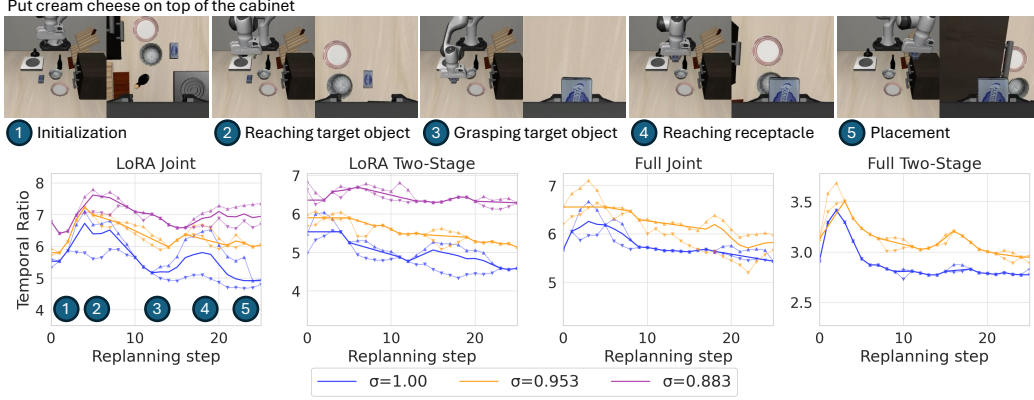


Figure 5: **Evolution of TR across episodes.** We illustrate the evolution of TR for each training category, for varying video noise levels and at $T = 5$. Upper triangular markers denote TR values for successful episodes while lower triangular markers denote TR values for failed episodes. It is evident that: (1) successful episodes are consistently characterized by higher TR and (2) TR peaks during the planning phase and drops during precise grasping or placement, shown using a rollout example for the task “put cream cheese on top of the cabinet.”

$$\text{Temporal Ratio: } \text{TR}_i^{(\ell)} = \frac{\sum_{q \in \mathcal{A}} \sum_{v \in \mathcal{V}_+} A_\ell(q, v)}{\sum_{q \in \mathcal{A}} \sum_{v \in \mathcal{V}_0} A_\ell(q, v)} = \frac{\text{Attention over future frames}}{\text{Attention over current frame}}, \quad (3)$$

where $\mathcal{A}$ denotes the action tokens. TR is high when actions rely on predicted futures and low when they anchor to the current observation. TR has a direct physical interpretation in VAMs, emerging as an artifact of the video denoising formulation: the first frame is injected as a clean conditioning frame at every video denoising step and serves as a high-fidelity observation anchor. Future frames, in contrast, are denoised from noisy latent targets, have lower signal-to-noise ratio (SNR) than the anchor, and represent the model’s imagined scene evolution. TR is useful because it is available from the policy’s normal forward pass and can be tracked online across replan steps. It therefore provides a runtime view of when the policy enters a predictive mode, rather than a post-hoc score computed only after an episode terminates.

4.1 TR Analysis in Simulation: Key Findings

We measure TR on LIBERO across tasks, execution timesteps, design configurations, and success outcomes. This lets us diagnose whether the policy is using the clean anchor frame as a reactive controller or using future latents as a plan.

TR increases with video denoising before feature extraction. More video denoising improves the SNR and semantic structure of generated rollouts. As shown in Fig. 5, the action head reflects this change: high-noise level video features suppress TR because future tokens are too unstructured, while lower-noise level video features encourage attention to imagined futures.

TR separates reactive and predictive phases of execution. TR also changes within a rollout: (1) It rises during planning moments such as object or receptacle selection, where the policy must bind language, objects, and goals through the predicted future. (2) It drops during local manipulation, where grasping and placement require precise current-frame grounding. Thus, low TR is not always harmful; it is appropriate when the task phase demands reactive action generation.

This motivates using TR to detect the policy’s execution phase at runtime and amplify compositional video conditioning primarily during predictive planning phases.

4.2 Adaptive TR Guidance

Motivated by previous findings, we propose a new inference-time adaptive guidance method based on TR. At each action denoising step, let v_{cond} be the standard conditional velocity. We compute two counterfactual directions that isolate complementary sources of compositional behavior.

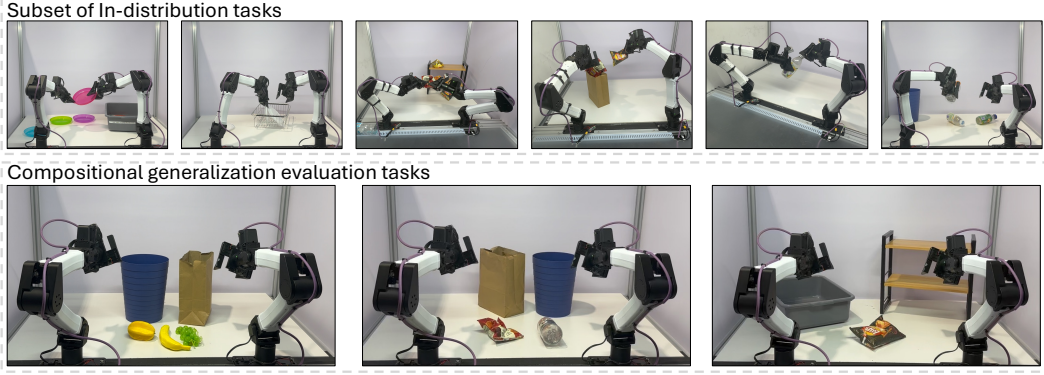


Figure 6: **Real-world setup.** (Top) We train on a bimanual YAM multi-task dataset with dish placement, snack sorting, bottle pick-and-place, and cap unscrewing. (Bottom) We show 3 test scenes with multiple target-object and receptacle candidates to evaluate compositional generalization.

179 **Language guidance:** We drop the text instruction ($c = \emptyset$) during video feature extraction and
 180 compute $v_{\text{uncond}} = v_{\phi}(a_{\sigma_a}, F_{\emptyset}^{(k)}, s, \sigma_a)$ where $F_{\emptyset}^{(k)} = F_{\theta}^{(k)}(x_{\sigma_v}, \emptyset, \sigma_v)$. A guidance direction in ac-
 181 tion flow-velocity can be formulated as $\Delta_{\text{lang}}^v = v_{\text{cond}} - v_{\text{uncond}}$ which isolates instruction-dependent
 182 action velocity and helps resolve ambiguous object-goal bindings.

183 **Plan guidance:** We query the video model with an extended horizon $T' > T$, truncate fea-
 184 tures back to the policy horizon T , and compute $v_{\text{plan}} = v_{\phi}(a_{\sigma_a}, F_{\text{plan}}^{(k)}, s, \sigma_a)$ where $F_{\text{plan}}^{(k)} =$
 185 $F_{\theta}^{(k)}(x_{\sigma_v}^{(T')}, c, \sigma_v)|_{\text{first } T}$. A plan guidance direction can be formulated as $\Delta_{\text{plan}}^v = v_{\text{plan}} - v_{\text{cond}}$ which
 186 isolates long-horizon temporal structure while preserving the fixed action-head input horizon.

187 The combined guided update becomes $v_{\text{guided}} = v_{\text{cond}} + w_{\text{lang},i} \Delta_{\text{lang}}^v + w_{\text{plan},i} \Delta_{\text{plan}}^v$. Rather than
 188 using fixed weights throughout the episode, we scale the guidance strengths with the current
 189 TR. For $w \in \{w_{\text{lang}}, w_{\text{plan}}\}$, the weights can be dynamically updated as $w_i = w^{\text{base}} s_i$ where
 190 $s_i = \text{clip}\left(\frac{\text{TR}_i}{\text{TR}_0} - 1, 0, s_{\text{max}}\right)$. Here, s_i (1) is zero when the current TR drops below the initial
 191 value TR_0 , (2) grows as the policy shifts toward future-latent reliance, and (3) is capped by s_{max}
 192 to avoid over-guidance. This concentrates intervention during planning phases and relaxes it dur-
 193 ing present-anchored manipulation, allowing guidance to improve compositional reasoning without
 194 hurting local precision while grasping or placement.

195 5 Experimental Results

196 The previous sections identify the failure mechanism behind the VAG gap: a VAM succeeds com-
 197 positionally only when the video backbone predicts a useful future and the action head follows that
 198 future during planning. This section tests whether that diagnosis translates into measurable gains by
 199 evaluating: (1) the VAG gap in prior WAMs and VAMs, (2) whether our design choices reduce it,
 200 and (3) whether TR-Adaptive Guidance improves OOD success without hurting ID performance.

201 **Datasets.** We use the four standard LIBERO suites [23]: libero-spatial, libero-object,
 202 libero-goal, and libero-long for simulation evaluation. Each suite contains 10 tasks, 50 demos
 203 per task. Following [10], the action head is trained on successful demos, while the video backbone
 204 finetuning uses the full unfiltered dataset. For real-world evaluation on bimanual YAM setup, we col-
 205 lect 5600 episodes across 24 tasks, including pick-and-place with dishes, snacks, bottles, and fruits;
 206 arranging objects on shelves; bimanual handover; t-shirt manipulation; and bottle-cap unscrewing.
 207 The multi-task dataset allows the policy to learn reusable object, receptacle, and manipulation fac-
 208 tors. Observations use three 224×168 RGB views (top, left wrist, right wrist) stacked horizontally,
 209 and the policy outputs a 14-dim bimanual action. More details are in App. E.

210 **Baselines in simulation and real-world study.** We compare against open-source VLAs (π_0 [24],
 211 $\pi_{0.5}$ [26]), WAMs (Cosmos-Policy [10], Fast-WAM [14]), and VAMs (Mimic-Video [12],
 212 DiT4DiT [13]). All WAM/VAM baselines use the same Cosmos-Predict 2.5-2B video backbone,

Table 1: LIBERO success rate (%) across ID and OOD suites. Baseline ID numbers are from original papers where available; OOD results use released checkpoints with 50 episodes per task per suite and 3 seeds. ID success is saturated, while OOD success exposes the VAG gap.

Category	Method	In-Distribution					Out-of-Distribution			
		Spatial	Object	Goal	Long	Avg ID	Spatial	Object	Goal	Avg OOD
VLA	π_0	96.8	98.8	95.8	85.2	94.2	0.7	0.3	4.3	1.7
	$\pi_{0.5}$	98.8	98.2	98.0	92.4	96.9	36.7	2.3	41.7	26.8
WAM/VAM	Cosmos-Policy	98.1	100.0	98.2	97.6	98.5	30.7	0.3	0.7	10.5
	Fast-WAM	98.2	100.0	97.0	95.2	97.6	12.7	0.0	15.7	9.4
	Mimic-Video	94.2	96.8	90.6	—	—	—	—	—	—
	DiT4DiT	98.4	99.6	98.6	97.6	98.6	9.0	0.0	10.3	6.4
	<i>Ours (best unguided)</i>	97.3	98.0	97.6	84.3	94.3	50.3	40.0	77.3	55.7
	<i>Ours (best guided)</i>	96.3	99.6	97.6	82.6	94.0	58.6	40.0	80.3	59.4

and the π_0 -family VLA baselines are at a similar 3B VLM scale. This comparison separates generic VLA pretraining from explicit world-action modeling that modifies the video generative prior, and from latent video conditioning with a separate action head. For LIBERO, we evaluate released checkpoints on the OOD suites. For real-world tasks, we train π_0 , $\pi_{0.5}$, and Cosmos-Policy on our bimanual YAM multi-task dataset with matched compute budgets: all equivalent to training for 1M steps on 32 NVIDIA® H200™ GPUs. More details are in App. E.

5.1 Compositional generalization evaluation testbed

LIBERO. We use the compositional OOD setup from Li [4]. These tasks recombine familiar objects, receptacles, layouts, and goals into unseen task sequences. The object and destination may each appear in training, but the connecting trajectory is novel, so success requires stitching sub-trajectories learned from disjoint ID tasks. `libero-goal-ood` and `libero-spatial-ood` test novel object-destination or layout-destination combinations while `libero-object-ood` requires semantic understanding of the object categories. More details are in App. B.

Bimanual YAM. We construct three OOD scenes with multiple target and receptacle candidates as shown in Fig. 6. Tasks 1–2 test unseen fruit-receptacle bindings and require selecting the correct receptacle. Tasks 3–6 require choosing between snacks/bottles and paper bag/bin, mixing seen and unseen pairings under a multi-receptacle shift. Tasks 7–8 test whether the policy overfits to the seen snack-shelf behavior when bin and shelf are both present. More details are in App. E.

5.2 Key Findings

The VAG gap appears across baselines. Tab. 1 compares ID and OOD success¹. ID performance is saturated for most methods ($> 95\%$), but compositional OOD success collapses for prior WAMs/VAMs. Among baselines, $\pi_{0.5}$ has the strongest OOD average, suggesting that strong VLA pretraining can reduce but not eliminate the gap. Our best unguided VAM extracts features at $\sigma_v = 0.883$ with horizon $T = 5$ (17 frames at 10 fps), while the action head predicts 16 actions at 10 Hz. This adds marginal compute while improving avg. OOD to 55.7%. Adaptive guidance further raises avg. OOD to 59.4%, more than $5\times$ prior VAM performance.

TR-based adaptive guidance mitigates the gap when video futures are useful. Guidance preserves most ID performance but slightly hurts LIBERO-long, indicating that stronger video conditioning can be harmful when predicted futures are unreliable. On compositional OOD suites, however, guidance improves avg. OOD by $\approx 6\%$. This supports the central mechanism: action-level generalization requires both a useful video rollout and sufficient action-head reliance on that rollout. If video features generalize but TR is low, the action head ignores the compositional plan; if TR is high but the video future is wrong, guidance can amplify the wrong behavior.

TR-based adaptive guidance boosts real-world performance on bimanual YAM. We transfer the best LIBERO design, LoRA backbone adaptation with joint training, to YAM. Video features

¹Mimic-Video [12] released checkpoints per ID task suite, making OOD comparison unfair.

Table 2: We compare the performance of baselines and our VAM with optimal design choices, with and without guidance, across 3 compositional evaluation scenes and 8 tasks. Tasks 1–2 are evaluated with 5 trials with 3 fruits (total 15 trials), and Tasks 3–8 are evaluated with 5 trials each. The tasks are designed such that a policy capable of precise manipulation but no generalization will have an avg. performance of 50%. Performance > 50% illustrates compositional generalization capabilities.

Method	Scene 1		Scene 2				Scene 3		Avg
	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6	Task 7	Task 8	
π_0	66.7	6.7	20.0	20.0	0.0	40.0	40.0	100.0	36.7
$\pi_{0.5}$	80.0	20.0	100.0	40.0	20.0	20.0	100.0	80.0	55.0
Cosmos-Policy	8.3	8.3	15.0	10.0	10.0	15.0	25.0	25.0	12.5
<i>Ours (unguided)</i>	80.0	53.3	100.0	60.0	40.0	100.0	100.0	60.0	71.7
<i>Ours (guided)</i>	80.0	73.3	100.0	80.0	80.0	100.0	100.0	80.0	83.3

are extracted at $\sigma_v = 0.883$ with horizon $T = 5$ (17 frames at 10 fps), while the action head predicts 50 actions at 30 Hz. Tab. 2 shows that our VAM outperforms baselines², and adaptive guidance further improves average success from 71.7% to 83.3%. Extracting features at the first denoising step ($\sigma_v = 1.0$), as in prior VAMs [12, 13], fails on all YAM tasks, highlighting the importance of structured, partially denoised video latents in multi-task VAMs.

6 Related Work

Generative video foundation models as backbone for policy learning. Generative video models [7, 8, 9] provide spatio-temporal priors absent from standard VLAs [28, 24, 26, 29, 30, 31, 32]. Robotics work transfers these priors through imagine-then-execute methods [33, 15, 34, 16, 35], WAMs that jointly model video and action [10, 11, 36, 17, 37, 38, 19, 20, 39, 40, 41, 42, 18], and latent VAMs that feed video features to an action head [43, 44, 12, 13]. Complementing Fast-WAM [14] and HarmoWAM [21], we study whether the action head actually uses future-predictive video features or retreats to present-state anchoring.

Studying compositional generalization for foundation-model-based policies. Compositional generalization [45, 46, 1, 2] is necessary because combinatorial demonstrations do not scale. Prior VLA studies identify spatial overfitting [4], modality imbalance [5], and brittle linguistic grounding [6, 3]; larger frontier systems reduce but do not remove the gap [29, 11]. We focus on video-conditioned policies, where the central question is whether generative temporal priors survive action finetuning and improve novel sequence compositions.

7 Conclusion

We study the VAG gap: when VAMs fail to inherit the compositional priors of their generative video backbones. Temporal Ratio (TR) reveals that successful compositional behavior appears when the action head shifts toward future-predictive latents during planning, then returns to present anchoring for local manipulation. TR also enables TR-Adaptive Guidance, an inference-time intervention that amplifies compositional video conditioning during planning phases. Across LIBERO and real-robot tasks, this framework clarifies VAM behavior and mitigates the OOD generalization gap. We will open-source our study codebase and trained models after double-blind review.

Limitations. Several limitations remain: (1) the presented study remains confined to a specific class of VAMs and is bottlenecked by the physical plausibility and compositional reach of the underlying video model, especially under distribution shifts where generated futures may become confident but infeasible. In the latter case, adaptive guidance will amplify incorrect guidance signals. (2) extra denoising steps and guidance add inference cost, reducing the inference frequency. More details are in App. E.

²Cosmos-Policy did not train well with the open-source codebase on our multi-task dataset despite sharing the same video backbone, suggesting that large multi-task WAM training remains an open problem.

References

- [1] K. Farid, R. Sahay, Y. A. Alnaggar, S. Schrodi, V. Fischer, C. Schmid, and T. Brox. What drives compositional generalization in visual generative models? *arXiv preprint arXiv:2510.03075*, 2025.
- [2] Y. Chen, Z. Chen, N. T. Chan, J. Chen, J. Yin, J. Shi, Y. Gao, Y.-L. Li, and J. Huo. Robo-himan: A hierarchical evaluation paradigm for compositional generalization in long-horizon manipulation. *arXiv preprint arXiv:2510.13149*, 2025.
- [3] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.
- [4] Q. Li. Task reconstruction and extrapolation for π_0 using text latent. *arXiv preprint arXiv:2505.03500*, 2025.
- [5] Y. Fang, Y. Feng, D. Jing, J. Liu, Y. Yang, Z. Wei, D. Szafir, and M. Ding. When vision overrides language: Evaluating and mitigating counterfactual failures in vlas. *arXiv preprint arXiv:2602.17659*, 2026.
- [6] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [7] A. Ali, J. Bai, M. Bala, Y. Balaji, A. Blakeman, T. Cai, J. Cao, T. Cao, E. Cha, Y.-W. Chao, et al. World simulation with video foundation models for physical ai. *arXiv preprint arXiv:2511.00062*, 2025.
- [8] T. Wan, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [9] Google DeepMind Veo Team. Veo: a text-to-video generation system. Technical report, Google DeepMind, 2025. URL <https://storage.googleapis.com/deepmind-media/veo/Veo-3-Tech-Report.pdf>.
- [10] M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn, et al. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026.
- [11] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [12] J. Pai, L. Achenbach, V. Montesinos, B. Forrai, O. Mees, and E. Nava. mimic-video: Video-action models for generalizable robot control beyond vlas. *arXiv preprint arXiv:2512.15692*, 2025.
- [13] T. Ma, J. Zheng, Z. Wang, C. Jiang, A. Cui, J. Liang, and S. Yang. Dit4dit: Jointly modeling video dynamics and actions for generalizable robot control. *arXiv preprint arXiv:2603.10448*, 2026.
- [14] T. Yuan, Z. Dong, Y. Liu, and H. Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- [15] H. Li, L. Sun, Y. Hu, D. Ta, J. Barry, G. Konidaris, and J. Fu. Novaflo: Zero-shot manipulation via actionable flow from generated videos. *arXiv preprint arXiv:2510.08568*, 2025.
- [16] Y. Du, S. Yang, P. Florence, F. Xia, A. Wahid, P. Sermanet, T. Yu, P. Abbeel, J. B. Tenenbaum, L. Kaelbling, et al. Video language planning. In *International Conference on Learning Representations*, volume 2024, pages 31138–31155, 2024.

- [17] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026.
- [18] J. Liang, P. Tokmakov, R. Liu, S. Sudhakar, P. Shah, R. Ambrus, and C. Vondrick. Video generators are robot policies. *arXiv preprint arXiv:2508.00795*, 2025.
- [19] S. Li, Y. Gao, D. Sadigh, and S. Song. Unified video action model. *arXiv preprint arXiv:2503.00200*, 2025.
- [20] C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.
- [21] Q. Feng, J. Yu, J. Liu, Y. Jia, Z. Wu, H. Chen, Z. Qian, S. Gu, P. Jia, S. Ma, et al. Harmowam: Harmonizing generalizable and precise manipulation via adaptive world action models. *arXiv preprint arXiv:2605.10942*, 2026.
- [22] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [23] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *ArXiv*, abs/2306.03310, 2023. URL <https://api.semanticscholar.org/CorpusID:259089508>.
- [24] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [25] G. Team. Gemma 3 technical report. *ArXiv*, abs/2503.19786, 2025. URL <https://api.semanticscholar.org/CorpusID:277313563>.
- [26] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [27] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [28] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [29] P. Intelligence, B. Ai, A. Amin, R. Aniceto, A. Balakrishna, G. Balke, K. Black, G. Bokinsky, S. Cao, T. Charbonnier, et al. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.
- [30] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [31] G. R. Team, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, T. Armstrong, A. Balakrishna, R. Baruch, M. Bauza, M. Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [32] H. Fang, J. Duan, D. Clay, S. Wang, S. Liu, W. Huang, X. Fan, W.-C. Tsai, S. Chen, Y. R. Wang, et al. Molmoact2: Action reasoning models for real-world deployment. *arXiv preprint arXiv:2605.02881*, 2026.

- [33] J. Jang, S. Ye, Z. Lin, J. Xiang, J. Bjorck, Y. Fang, F. Hu, S. Huang, K. Kundalia, Y.-C. Lin, et al. Dreamgen: Unlocking generalization in robot learning through video world models. *arXiv preprint arXiv:2505.12705*, 2025.
- [34] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, D. Schuurmans, and P. Abbeel. Learning universal policies via text-guided video generation. *Advances in neural information processing systems*, 36:9156–9172, 2023.
- [35] P.-C. Ko, J. Mao, Y. Du, S.-H. Sun, and J. B. Tenenbaum. Learning to act from actionless videos through dense correspondences. In *International Conference on Learning Representations*, volume 2024, pages 40938–40958, 2024.
- [36] B. Chen, T. Zhang, H. Geng, C. Zhang, P. Li, K. Song, W. T. Freeman, J. Malik, P. Abbeel, R. Tedrake, et al. Large video planner enables generalizable robot control. *arXiv preprint arXiv:2512.15840*, 2025.
- [37] C. Deng, D. Zhu, K. Li, C. Gou, F. Li, Z. Wang, S. Zhong, W. Yu, X. Nie, Z. Song, et al. Emerging properties in unified multimodal pretraining. *arXiv preprint arXiv:2505.14683*, 2025.
- [38] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, volume 2024, pages 10641–10662, 2024.
- [39] R. Zheng, J. Wang, S. Reed, J. Bjorck, Y. Fang, F. Hu, J. Jang, K. Kundalia, Z. Lin, L. Magne, et al. Flare: Robot learning with implicit world modeling. *arXiv preprint arXiv:2505.15659*, 2025.
- [40] Q. Zhao, Y. Lu, M. J. Kim, Z. Fu, Z. Zhang, Y. Wu, Z. Li, Q. Ma, S. Han, C. Finn, et al. Cotvla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1702–1713, 2025.
- [41] B. Tharwat, Y. Nasser, A. Abouzeid, and I. Reid. Latent action pretraining through world modeling. *arXiv preprint arXiv:2509.18428*, 2025.
- [42] Y. Hu, Y. Guo, P. Wang, X. Chen, Y.-J. Wang, J. Zhang, K. Sreenath, C. Lu, and J. Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *arXiv preprint arXiv:2412.14803*, 2024.
- [43] H. Qi, H. Yin, A. Zhu, Y. Du, and H. Yang. Inference-time enhancement of generative robot policies via predictive world modeling. *IEEE Robotics and Automation Letters*, 11(5):5534–5541, 2026. doi:10.1109/LRA.2026.3673995.
- [44] Y. Luo and Y. Du. Grounding video models to actions through goal conditioned exploration. In *International Conference on Learning Representations*, volume 2025, pages 92200–92232, 2025.
- [45] M. Okawa, E. S. Lubana, R. P. Dick, and H. Tanaka. Compositional abilities emerge multiplicatively: Exploring diffusion models on a synthetic task. *ArXiv*, abs/2310.09336, 2023. URL <https://api.semanticscholar.org/CorpusID:264146105>.
- [46] T. Wiedemer, P. Mayilvahanan, M. Bethge, and W. Brendel. Compositional generalization from first principles. *Advances in Neural Information Processing Systems*, 36:6941–6960, 2023.
- [47] I2RT. YAM 6-DOF arm. <https://i2rt.com/products/yam-6-dof-arm>, 2025. Accessed 2026-06-03.
- [48] Intel. Intel RealSense Stereo Depth Camera D405. <https://www.realsenseai.com/products/stereo-depth-camera-d405/>, 2025. Accessed 2026-06-03.

412	Contents	
413	1 Introduction	1
414	2 Design Space of Video-Action Models	2
415	2.1 Background	2
416	2.2 Design Axes	3
417	3 Systematic Analysis of the Video-Action Generalization Gap	3
418	4 Understanding and Mitigating the VAG Gap via Temporal Ratio	4
419	4.1 TR Analysis in Simulation: Key Findings	5
420	4.2 Adaptive TR Guidance	5
421	5 Experimental Results	6
422	5.1 Compositional generalization evaluation testbed	7
423	5.2 Key Findings	7
424	6 Related Work	8
425	7 Conclusion	8
426	A Architecture and Algorithm Details	14
427	A.1 Latent VAM Architecture	14
428	A.2 Training and Inference Procedure	14
429	A.3 TR-Adaptive Guidance Algorithm	16
430	B Simulation Task Setup	17
431	B.1 LIBERO In-Distribution Suites	17
432	B.2 LIBERO Compositional OOD Suites	17
433	B.3 Relation to Other LIBERO Extensions	18
434	C Design-Space Analysis and Additional Results	18
435	C.1 Full Training, Noise-Level, and Horizon Sweep	18
436	C.2 Per-Suite Breakdown	19
437	C.3 Video Feature Extraction Layer Ablation	19
438	C.4 Gemma Layer selection for TR	20
439	D Implementation and Hyperparameter Details	20
440	D.1 Video Backbone Adaptation	20
441	D.2 Action Head Training	21
442	D.3 Feature Extraction	22

443	D.4 Guidance Hyperparameters	23
444	E Real-World Setup and Evaluation Details	23
445	E.1 Hardware Description	23
446	E.2 Bimanual YAM Platform	24
447	E.3 Real-World Training Dataset: PP24	24
448	E.4 Real-World Compositional Evaluation	24
449	E.5 Real-World Training Hyperparameters and Compute	25
450	E.6 Inference Cost	26

451 All videos are included in supplementary and <https://temporal-ratio.github.io/>.

452 A Architecture and Algorithm Details

453 A.1 Latent VAM Architecture

454 **Video backbone.** We instantiate F_θ as the 28-block Cosmos-Predict2.5-2B DiT [7] (~ 2.1 B parameters). Inputs are VAE-encoded video latents from the Cosmos-Tokenzer ($4 \times$ temporal and $8 \times$ spatial compression), patchified by a Rearrange + Linear embedder with patch size $1 \times 2 \times 2$, and position-encoded with 3D rotary embeddings (rope3d). Each block contains self-attention over video tokens, cross-attention to text, a linear module ($2048 \rightarrow 8192 \rightarrow 2048$), and Adaptive Layer-Norm (shift/scale/gate) conditioned on the diffusion timestep. Self-attention is 2048-dim with 16 heads; cross-attention queries from video (2048) and projects keys/values from text ($1024 \rightarrow 2048$). Text is encoded by Cosmos-Reason1-7B (Qwen2.5-VL-7B), giving task prompt embeddings of shape $(B, 512, 100352)$ that are projected once through a shared linear layer ($100352 \rightarrow 1024$) and reused as the same `crossattn_emb` in all 28 blocks (no per-block text modulation).

464 **Action head.** The action head π_ϕ is the 18-layer Gemma expert from π_0 [24, 26, 25] (~ 300 M parameters, hidden dim 1024). Attention is multi-query: 8 query heads and 1 shared key/value head with head dimension 256. Each transformer layer uses a GeGLU MLP (`gate_proj`, `up_proj`, `down_proj` via `gelu_pytorch_tanh`) and Adaptive RMSNorm whose dense projection $\mathbb{R}^{1024} \rightarrow \mathbb{R}^{3072}$ produces a shift/scale/gate triple conditioned on the action diffusion timestep σ_a , with the gate modulating the residual contribution of each sublayer. The token sequence concatenates flattened video tokens (per latent frame), a single state token, and 17 noisy action tokens; attention follows a prefix-LM mask [24]: bidirectional within the video prefix, causal at the state and first-action boundaries, and bidirectional among the remaining action tokens.

473 **Video-feature interface.** At inference, we extract $F_\theta^{(k)}(x_{\sigma_v}, c, \sigma_v) \in \mathbb{R}^{T \times (H \times W) \times D}$ with $k=20$, $D=2048$, and $(H, W)=(14, 28)$ from the 448×224 observation (primary + wrist camera, concatenated horizontally). Each spatial $1 \times 2 \times 2$ neighborhood (4 tokens, $4 \times 2048=8192$ features) is projected to the action-head dimension (1024) by a linear layer, yielding 98 video tokens per latent frame. The video noise level σ_v is encoded with a Fourier feature MLP and applied once through a video-only Adaptive layer norm (shift+scale, no gate) before the tokens enter the Gemma transformer, so that each video token carries an explicit “noise level” tag. Proprioception (8-dim) is padded to 32 following π -models [24] and then embedded to 1024 via `state_proj`; noisy action chunks ($17 \text{ steps} \times 7 \text{ dims}$, internally padded to dimension 32) are likewise embedded as well, and then finally decoded into original dimensions to get the flow velocity output. The language instruction is consumed only by the video backbone via cross-attention; the action head sees video features, state, and the action timestep alone.

485 A.2 Training and Inference Procedure

486 **Video denoising.** The video backbone is trained with the rectified-flow loss of Eq. 1 under $\sigma_v \sim \mathcal{T}_v = \mathcal{U}[0, 1]$, predicting velocity $\epsilon_v - x_0$ from $x_{\sigma_v} = (1 - \sigma_v)x_0 + \sigma_v \epsilon_v$. At inference we discretize the flow integral $x_0 = x_1 + \int_1^0 v_\theta(x_{\sigma'_v}, c, \sigma'_v) d\sigma'_v$ with a multistep predictor-corrector solver (FlowUniPC, shift 5.0) using 6 steps; the shift concentrates the schedule near $\sigma_v=1$ so that the model spends most of its iterations on the high-noise regime. Features are exposed at one chosen step on this 6-point grid, and we sweep that index over $\{0, 1, 2, 4, 5\}$, corresponding to $\sigma_v \in \{1.000, 0.953, 0.883, 0.562, 0.024\}$.

493 **Action denoising.** The action head is trained with the flow-matching loss $\mathcal{L}_{\text{action}} = \mathbb{E}_{a_0, \epsilon_a, \sigma_a} [\|v_\phi(a_{\sigma_a}, F_\theta^{(k)}, s, \sigma_a) - (\epsilon_a - a_0)\|^2]$ under $\sigma_a \sim \mathcal{T}_a$, where $\mathcal{T}_a$ is the beta-distribution $\text{Beta}(1.5, 1.0) \cdot 0.999 + 0.001$ (following [24]); this favours high action noise during training, matching the inference initial state $a_{\sigma_a=1} \sim \mathcal{N}(0, I)$. The target is the constant-conditional velocity $\epsilon_a - a_0$.

497 At inference we integrate v_ϕ from $\sigma_a=1$ to 0 with 10 Euler steps on a single fixed action chunk of
 498 length 17. The video features $F_\theta^{(k)}(x_{\sigma_v}, c, \sigma_v)$ are precomputed once per replan and reused across all
 499 10 Euler steps, so the cost is dominated by the single video pass.

Algorithm 1 Joint Training of F_θ and v_ϕ

Require: Extraction layer k , flow-time distributions $\mathcal{T}_v, \mathcal{T}_a$, video loss weight λ

```

1: repeat
2:   Sample  $(x_0, a_0, s, c) \sim \mathcal{D}$  ▷ video latent, action chunk, state, instruction
3:    $\sigma_v \sim \mathcal{T}_v; \quad \sigma_a \sim \mathcal{T}_a$ 
4:    $\epsilon_v, \epsilon_a \sim \mathcal{N}(0, I)$ 
5:    $x_{\sigma_v} \leftarrow (1 - \sigma_v)x_0 + \sigma_v \epsilon_v$  ▷  $t=0$  frame re-injected as clean
6:    $a_{\sigma_a} \leftarrow (1 - \sigma_a)a_0 + \sigma_a \epsilon_a$ 
7:    $F^{(k)} \leftarrow F_\theta^{(k)}(x_{\sigma_v}, c, \sigma_v)$  ▷ forward video DiT up to block  $k$ 
8:    $\mathcal{L}_{\text{video}} \leftarrow \|v_\theta(x_{\sigma_v}, c, \sigma_v) - (\epsilon_v - x_0)\|^2$ 
9:    $\mathcal{L}_{\text{action}} \leftarrow \|v_\phi(a_{\sigma_a}, F^{(k)}, s, \sigma_a) - (\epsilon_a - a_0)\|^2$ 
10:  Take gradient step on  $\mathcal{L}_{\text{action}} + \lambda \mathcal{L}_{\text{video}}$  w.r.t.  $\theta, \phi$ 
11: until converged

```

500 Our training procedure is similar to [12, 13]. In two-stage training we instantiate Algorithm 1 twice
 501 with disjoint trainable parameters. *Stage 1* trains only the video backbone by optimising $\mathcal{L}_{\text{video}}$ alone
 502 (lines 7–8) with ϕ ignored; *Stage 2* freezes θ , caches $F^{(k)}$ at randomly sampled σ_v , and trains ϕ on
 503 $\mathcal{L}_{\text{action}}$ alone. Joint training keeps both losses active, with $\lambda=1.0$ used throughout.

Algorithm 2 Action Sampling at replan step i

Require: Extraction layer k , target video flow-time σ_v^* , action denoising steps N_a

```

1: Input: observation  $o_t$ , proprioceptive state  $s$ , language instruction  $c$ 
2:  $x_{\sigma_v=1} \sim \mathcal{N}(0, I); \quad a_{\sigma_a=1} \sim \mathcal{N}(0, I)$  ▷ anchor  $t=0$  in  $x$  replaced by clean  $o_t$ 
3:  $x_{\sigma_v^*} \leftarrow x_{\sigma_v=1} + \int_1^{\sigma_v^*} v_\theta(x_{\sigma_v'}, c, \sigma_v') d\sigma_v'$  ▷  $\approx$  6-step FlowUniPC, truncated at  $\sigma_v^*$ 
4:  $F^{(k)} \leftarrow F_\theta^{(k)}(x_{\sigma_v^*}, c, \sigma_v^*)$  ▷ partial denoising avoids full pixel reconstruction
5:  $a_0 \leftarrow a_{\sigma_a=1} + \int_1^0 v_\phi(a_{\sigma_a'}, F^{(k)}, s, \sigma_a') d\sigma_a'$  ▷  $N_a=10$  Euler steps
6: return action chunk  $a_0$ 

```

504 **First-frame anchor mechanism during video denoising.** The $t=0$ latent encodes the cur-
 505 rent image observation o_t and is held *exactly clean* throughout video denoising. The Cosmos-
 506 Predict2.5 backbone (<https://github.com/nvidia-cosmos/cosmos-predict2.5>) uses three
 507 coordinated operations. (i) *Anchor channel:* A binary mask $M \in \{0, 1\}^{B \times 1 \times T \times H \times W}$ with $M[:, :, 0] = 1$
 508 and zeros elsewhere is concatenated as an extra channel to the 16-channel VAE latent, so the DiT
 509 receives a 17-channel input that marks which positions are anchors. (ii) *Pre-network latent re-*
 510 *placement:* Before every solver call, the noisy latent is overwritten at the anchor positions with the
 511 clean observation latent $x_0^{(t=0)}$: $x_{\sigma_v} \leftarrow M \odot x_0^{(t=0)} + (1-M) \odot x_{\sigma_v}$. The DiT therefore never sees a
 512 noised current frame at any σ_v . (iii) *Post-network velocity replacement:* After the DiT predicts v_θ ,
 513 the predicted velocity at anchor positions is overwritten with the analytic “stay-at-clean” velocity
 514 $\epsilon_v - x_0^{(t=0)}$, so the subsequent Euler/UniPC update lands back on the clean anchor. When configured,
 515 the per-position timestep embedding at anchor positions is also replaced with a near-zero constant
 516 ($\sigma_v=10^{-4}$), which feeds the “clean” signal into the adaptive LayerNorm conditioning.

517 **Significance for Temporal-Ratio.** The above described asymmetry is what makes TR informative.
 518 First frame tokens ($\mathcal{V}_0$) carry a high-SNR, language-independent observation; future frame tokens
 519 ($\mathcal{V}_+$) carry the model’s noisy imagined rollout, whose SNR is set by σ_v . The attention the action
 520 head puts on $\mathcal{V}_0$ versus $\mathcal{V}_+$ is therefore a routing decision between reacting to the current image
 521 and following the imagined future. The monotonic increase of TR as σ_v drops, which we report in
 522 Sec. 4.1, follows directly from holding the anchor clean.

Algorithm 3 TR-Adaptive Velocity-Space Guidance at replan step i

Require: Observation o_t , proprioceptive state s , language instruction c , target video flow-time σ_v^* , action denoising steps N_a

Require: Base weights $w_{\text{lang}}^{\text{base}}, w_{\text{plan}}^{\text{base}}$, schedule $\in \{\text{flat}, \text{adaptive}\}$, cap s_{max}

Require: Plan horizon $T' > T$, TR layer ℓ^* , persistent episode baseline TR_0 (initialised at the first replan, $i=0$)

- 1: **// Video feature branches (each: 6-step FlowUniPC truncated at σ_v^*)**
- 2: $F^{(k)} \leftarrow F_{\theta}^{(k)}(x_{\sigma_v^*}, c, \sigma_v^*)$ ▷ conditional
- 3: **if** $w_{\text{lang}}^{\text{base}} > 0$ **then**
- 4: $F_{\varnothing}^{(k)} \leftarrow F_{\theta}^{(k)}(x_{\sigma_v^*}, \varnothing, \sigma_v^*)$ ▷ null-text
- 5: **end if**
- 6: **if** $w_{\text{plan}}^{\text{base}} > 0$ **then**
- 7: $F_{\text{plan}}^{(k)} \leftarrow F_{\theta}^{(k)}(x_{\sigma_v^*}^{T'}, c, \sigma_v^*)|_{\text{first } T}$ ▷ extended-horizon, truncated to T
- 8: **end if**
- 9:
- 10: **// Read TR once per replan and update adaptive scale**
- 11: $\text{TR}_i \leftarrow \text{TR}$ at layer ℓ^* of $v_{\phi}(\cdot, F^{(k)}, s, \sigma_a=1)$ ▷ from action-head attention; Eq. 3
- 12: **if** $i = 0$ **then** $\text{TR}_0 \leftarrow \text{TR}_i$ ▷ episode baseline; reset only on episode reset, not every replan
- 13: **end if**
- 14: **if** schedule = flat **then**
- 15: $w_{\text{lang},i} \leftarrow w_{\text{lang}}^{\text{base}}, w_{\text{plan},i} \leftarrow w_{\text{plan}}^{\text{base}}$
- 16: **else** ▷ TR-adaptive
- 17: $s_i \leftarrow \text{clip}(\text{TR}_i / \text{TR}_0 - 1, 0, s_{\text{max}})$
- 18: $w_{\text{lang},i} \leftarrow w_{\text{lang}}^{\text{base}} \cdot s_i, w_{\text{plan},i} \leftarrow w_{\text{plan}}^{\text{base}} \cdot s_i$
- 19: **end if**
- 20:
- 21: **// Velocity-space CFG inside the action Euler loop**
- 22: $a_{\sigma_a=1} \sim \mathcal{N}(0, I)$
- 23: **for** Euler step $n = 1, \dots, N_a$ with $\sigma_a : 1 \rightarrow 0$ **do**
- 24: $v_{\text{cond}} \leftarrow v_{\phi}(a_{\sigma_a}, F^{(k)}, s, \sigma_a)$
- 25: **if** $w_{\text{lang},i} > 0$ **then**
- 26: $\Delta_{\text{lang}}^v \leftarrow v_{\text{cond}} - v_{\phi}(a_{\sigma_a}, F_{\varnothing}^{(k)}, s, \sigma_a)$
- 27: **else**
- 28: $\Delta_{\text{lang}}^v \leftarrow 0$
- 29: **end if**
- 30: **if** $w_{\text{plan},i} > 0$ **then**
- 31: $\Delta_{\text{plan}}^v \leftarrow v_{\phi}(a_{\sigma_a}, F_{\text{plan}}^{(k)}, s, \sigma_a) - v_{\text{cond}}$
- 32: **else**
- 33: $\Delta_{\text{plan}}^v \leftarrow 0$
- 34: **end if**
- 35: $v_{\text{guided}} \leftarrow v_{\text{cond}} + w_{\text{lang},i} \Delta_{\text{lang}}^v + w_{\text{plan},i} \Delta_{\text{plan}}^v$
- 36: Euler-update a_{σ_a} using v_{guided}
- 37: **end for**
- 38: **return** action chunk a_0

524 TR-adaptive guidance (Algorithm 3) reuses the same forward pass as the standard denoising: the
 525 conditional velocity $v_{\phi}(\cdot, F_{\theta}^{(k)}, s, \sigma_a)$ at the first Euler step is logged through the action head's at-
 526 tention, the TR is read off at layer ℓ^* , and the per-step weights $w_{\text{lang}}, w_{\text{plan}}$ are rescaled before the
 527 remaining Euler steps. Because $a_{\sigma_a=1}$ is freshly sampled each replan, the TR baseline TR_0 is reset
 528 at $i=0$ of every episode, not every replan.

B Simulation Task Setup

B.1 LIBERO In-Distribution Suites

Suite definitions. We use the four standard LIBERO [23] suites: `libero_spatial`, `libero_object`, `libero_goal`, and `libero_10` (also `libero_long`, for long-horizon tasks). Each suite has 10 tasks. The action space is 7-dim end-effector pose (6 pose deltas +1 gripper) and the observation is two 224×224 RGB cameras (`agentview` and `eye_in_hand`) with an 8-dim proprioception.

Training demonstrations. Each task ships with 50 human demonstrations (500 per suite). We use the filtered demonstrations from <https://huggingface.co/datasets/physical-intelligence/libero> without any modifications and concatenate the two cameras into the 448×224 input the video backbone expects. The same set trains every paradigm (joint, two-stage, LoRA, full FT).

Evaluation protocol. We use two evaluation budgets. The headline baseline comparison (Table 1) runs 50 rollouts per task across 3 trial seeds: 1,500 per suite, 10,500 total. The design-space grid sweep (Sec. 4.2) drops to 10 rollouts per task at 3 trial seeds to keep the 60-cell sweep tractable, giving 700 rollouts per configuration. Trial seeds hash deterministically over (task, episode-index, seed-id), so reruns reproduce the initial states. The policy runs at 10Hz (App. A.2). Max episode length is 220, 280, 300, 520 steps on `spatial`, `object`, `goal`, `libero_10`, and $1.5 \times$ those on the OOD counterparts (330, 420, 450).

Table 3: LIBERO evaluation suites. Budget shown is for the headline baseline comparison; the grid sweep uses 10 rollouts per task at 3 trial seeds.

Suite	Dist.	Tasks	Ep./task	Seeds	Total ep.
Spatial	ID	10	50	3	1500
Object	ID	10	50	3	1500
Goal	ID	10	50	3	1500
Long	ID	10	50	3	1500
Spatial-OOD	OOD	10	50	3	1500
Object-OOD	OOD	10	50	3	1500
Goal-OOD	OOD	10	50	3	1500
Total per config.		70	—	3	10500

B.2 LIBERO Compositional OOD Suites

The three OOD suites are from Li [4] <https://github.com/Quanyili/pi0-text-latent>. They modify the LIBERO BDDL files along one controlled axis at a time. Camera, simulator, robot, and scene match the corresponding ID suite; only the language–object–goal binding moves.

Spatial-OOD (object axis). Four tasks use new pick targets (`butter`, `chocolate_pudding`, `milk`, `orange_juice`) with the familiar `plate`. Six use the familiar `akita_black_bowl` placed via spatial relations not seen in the ID set (on `cookie_box`, `next_to` `plate`, at `table_center`, each onto `stove` or `cabinet`). The predicate vocabulary (`on`, `next_to`, `between`) is the same as in training.

Object-OOD (identity axis). Task names, scene, object positions, and goal predicates match `libero_object`; the BDDL only swaps which type is bound to each named instance. For example, `pick_up_the_butter_and_place_it_in_the_basket.bddl` declares `butter_1` – `chocolate_pudding` and `chocolate_pudding_1` – `butter`, and the goal binds `chocolate_pudding_1` (the instance that visually looks like butter). A policy that memorised

position-to-name fails; only one that grounds language in the current scene succeeds. The swap covers all ten classes.

Goal-OOD (composition axis). The object inventory matches `libero_goal`, but each task pairs an object with a destination it never reaches in ID. The ten pairings include `wine_bottle` $\rightarrow$ {stove, plate, bowl}, `cream_cheese` $\rightarrow$ {basket, plate, stove, top-of-cabinet}, `orange_juice` $\rightarrow$ stove, `bbq_sauce` $\rightarrow$ plate, and `tomato_sauce` $\rightarrow$ top-of-cabinet. Both sub-skills exist in ID tasks; the binding does not. Success requires stitching trajectories from disjoint ID tasks.

B.3 Relation to Other LIBERO Extensions

Two recent LIBERO extensions overlap with the suites we use. LIBERO-Plus [6] adds 10,030 tasks across seven perturbation dimensions: camera viewpoint, robot pose, lighting, textures, sensor noise, distractors, and paraphrased instructions. LIBERO-PRO [3] adds five: object appearance/colour/scale, position, paraphrase, task-logic edits, and environment swaps. Both report strong baselines dropping from ~ 0.95 ID success to < 0.3 under their perturbations. LIBERO-PRO’s task-logic dimension is the closest analogue of goal-OOD: both replace the goal predicate with a binding absent from training.

The benchmarks are complementary. LIBERO-Plus and LIBERO-PRO sweep many dimensions in one benchmark, mixing perceptual and compositional shifts while LIBERO-OOD’s three suites all sit on the language–object–goal binding axis. The Temporal Ratio analysis of Sec. 4.1 probes that axis, which is why we use LIBERO-OOD here.

C Design-Space Analysis and Additional Results

C.1 Full Training, Noise-Level, and Horizon Sweep

Finetuning strategy. LoRA dominates Full FT on OOD across every cell of the sweep. Joint + LoRA peaks at 55.7% OOD (Spatial peak $\sigma_v=0.883$, $T=5$); Two-Stage + LoRA peaks at 51.0% OOD. Full FT show competitive ID performance (Two-Stage + Full reaches 95.2% ID at $\sigma_v=0.883$, $T=5$) but lacks OOD generalization (33.0% OOD SR). The pattern is consistent with full-DiT updates destroying the pretrained video prior that supports OOD recomposition.

Video noise level. For Joint + LoRA at $T=5$, both ID and OOD peak at the same intermediate noise level $\sigma_v=0.883$ (94.3% ID, 55.7% OOD); pure noise ($\sigma_v=1.0$) costs 16 pp on ID and 15 pp on OOD, and the near-clean extraction ($\sigma_v=0.024$) costs another 12 pp on ID. The non-monotonic pattern repeats for the other paradigms: every $T=5$ row hits its joint ID/OOD optimum at $\sigma_v \in \{0.883, 0.953\}$. Two-Stage models are more robust to noise level and maintain performance at low video noise levels ($\sigma_v=0.024$).

Prediction horizon. The horizon ablation only changes how far the video DiT rolls out at evaluation time. *Training is always done with a $T=5$ latent-frame prediction target*, and at evaluation we always pass the first 5 latent frames of the video rollout to the action head, irrespective of weight video denoising happens for 5, 9, or 24 latent frames. Increasing the imagined rollout horizon therefore only changes the quality of the first 5 latent frames (longer video horizon allows the video model to maintain object-goal relationships), not the number of frames seen by the action head. With that setup, $T=5$ at evaluation matches the training distribution and dominates every paradigm at every σ_v . Stretching the evaluation rollout to $T=9$ costs 20–40 pp on most cells; $T=24$ collapses Joint variants almost entirely. Similar to video-noise level study, Two-Stage models do not hurt the video prior at all and helps in maintaining imagined rollout’s physical plausibility. The asymmetric collapse mirrors the failure mode in Sec. 4.2: while longer rollouts preserve the object-goal relationship, they incorporate more hallucinations like frame jumping and implausible transitions.

Table 4: Baseline SR (%) by training paradigm, denoising target step, and horizon on LIBERO. LoRA rows use $r=32$.

Paradigm	Horizon	$\sigma \approx 1.0$		$\sigma \approx 0.953$		$\sigma \approx 0.883$		$\sigma \approx 0.562$		$\sigma \approx 0.024$	
		ID	OOD	ID	OOD	ID	OOD	ID	OOD	ID	OOD
Joint + LoRA	$T=5$	78.0	41.0	88.0	53.7	94.3	55.7	83.5	49.7	82.5	45.3
	$T=9$	69.5	34.3	56.2	40.3	49.2	32.0	49.5	28.3	43.2	25.0
	$T=24$	13.8	11.3	5.8	9.7	0.0	2.7	0.2	2.3	0.0	1.0
Joint + Full	$T=5$	72.0	23.0	88.5	29.3	93.7	33.0	88.2	24.7	82.5	18.7
	$T=9$	42.0	8.3	47.0	11.3	42.0	14.0	45.0	11.3	33.8	6.7
	$T=24$	3.0	0.3	7.5	5.3	3.5	3.7	1.2	1.3	1.0	1.0
Two-Stage + LoRA	$T=5$	88.0	38.7	95.0	51.0	96.5	46.0	95.8	40.7	92.2	42.3
	$T=9$	61.8	22.0	71.2	37.0	69.0	39.7	71.2	37.3	71.5	41.3
	$T=24$	13.5	10.0	28.2	26.7	22.0	25.7	22.8	26.3	21.0	28.7
Two-Stage + Full	$T=5$	90.0	11.3	92.5	22.0	95.2	29.7	94.2	29.0	93.0	29.7
	$T=9$	78.0	4.7	78.2	10.3	74.8	13.3	64.2	17.0	58.5	15.0
	$T=24$	63.5	1.7	48.5	7.0	33.5	11.7	20.2	12.7	16.2	12.7

607 C.2 Per-Suite Breakdown

Table 5: Per-suite ID SR (%) for the unguided baseline, flat guidance, and TR-adaptive guidance. Combined guidance uses $w_{\text{lang}}=0.1$, $w_{\text{plan}}=0.3$, plan horizon $T'=24$.

Configuration	Setting	Spatial	Object	Goal	Long	Avg ID
Ours (unguided)	$\sigma_v=0.883$, $T=5$	97.3	98.0	97.6	84.3	94.3
Ours (flat guided)	combined, flat schedule	85.6	90.6	88.3	72.3	84.3
Ours (adaptive guided)	combined, TR-adaptive	96.3	99.6	97.6	82.6	94.0

Table 6: Per-suite OOD SR (%) for the unguided baseline, flat guidance, and TR-adaptive guidance

Configuration	Setting	Spatial	Object	Goal	Avg OOD
Ours (unguided)	$\sigma_v=0.883$, $T=5$	50.3	40.0	77.3	55.7
Ours (flat guided)	combined, flat schedule	53.6	40.0	78.3	57.3
Ours (adaptive guided)	combined, TR-adaptive	58.6	40.0	80.3	59.4

608 **Flat versus TR-adaptive scheduling.** Both schedules use the same combined guidance signal
609 ($w_{\text{lang}}=0.1$, $w_{\text{plan}}=0.3$); they differ only in how the per-step weight is set. Flat applies the base
610 weight at every action Euler step; TR-adaptive multiplies it by $s_i = \text{clip}(\text{TR}_i/\text{TR}_0 - 1, 0, s_{\text{max}})$, so
611 the guidance weight increases only when TR rises above its episode baseline (Alg. 3). While both
612 schedules help in improving OOD success by employing more compositional video guidance from
613 the backbone, flat guidance applies guidance even during the precise manipulation phases. This
614 makes grasping inaccurate and flat-guidance leads to a significant loss in ID performance. The
615 TR analysis of Sec. 4.1 predicts this split: planning-heavy phases of an episode have high TR and
616 benefit from amplified video conditioning, while precise-manipulation phases have low TR and need
617 to react to the current frame. TR-adaptive scheduling concentrates the guidance into the planning
618 phases without interfering with the precise manipulation phases.

619 C.3 Video Feature Extraction Layer Ablation

620 **DiT block selection.** We sweep the extraction layer over $k \in \{3, 6, 16, 20, 28\}$ (28-block DiT, 1-
621 indexed) under two noise levels: a high-noise extraction ($\text{inf}=1$, $\sigma_v \approx 0.99$) and a low-noise ex-
622 traction ($\text{inf}=4$, $\sigma_v \approx 0.562$). Each cell is averaged across the four training paradigms (Joint $\times$
623 LoRA/Full, Two-Stage $\times$ LoRA/Full) at latent horizon $T=5$. Results are summarised in Table 7

and per-suite breakdown is provided in Fig. 7. $k=20$ is the joint optimum across ID and OOD and aligns with the findings of [12, 13]; layers shallower than $k=16$ are clearly suboptimal under both noise levels.

Table 7: Effect of DiT extraction layer k on Avg ID / Avg OOD SR (%). Averaged across the four training paradigms at latent horizon $T=5$. $\text{inf}=1$ corresponds to the high-noise regime ($\sigma_v \approx 0.99$), $\text{inf}=4$ to the low-noise regime ($\sigma_v \approx 0.562$).

	$k=3$		$k=6$		$k=16$		$k=20$		$k=28$	
Noise level	ID	OOD	ID	OOD	ID	OOD	ID	OOD	ID	OOD
$\text{inf}=1$ ($\sigma_v \approx 0.99$)	55.5	0.0	58.5	0.0	87.8	2.3	94.3	7.0	90.0	7.7
$\text{inf}=4$ ($\sigma_v \approx 0.562$)	77.5	2.0	81.3	6.3	82.8	20.3	84.5	21.0	81.3	19.7

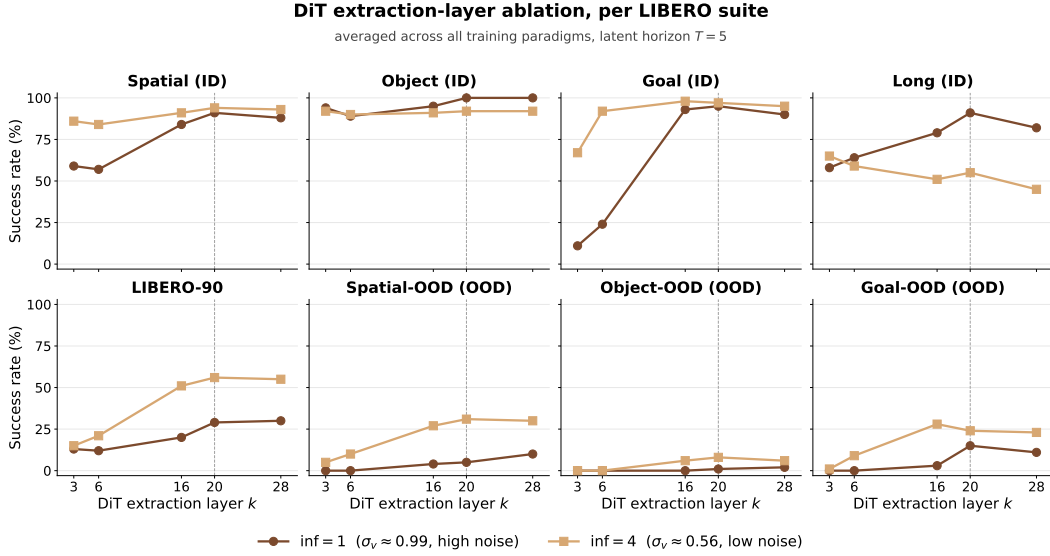


Figure 7: DiT extraction layer ablation across the seven LIBERO suites plus LIBERO-90, averaged across the four training paradigms at $T=5$. The two noise levels probe the same layer grid: $\text{inf}=1$ ($\sigma_v \approx 0.99$, high noise) and $\text{inf}=4$ ($\sigma_v \approx 0.562$, low noise). $k=20$ is the joint optimum used throughout the rest of the paper.

C.4 Gemma Layer selection for TR

We choose Gemma layer $\ell^* = 15$ for all reported TR values. The choice depends on how much each layer’s attention split between the current frame and the predicted future frames moves as the extraction noise level σ_v changes. Fig. 8 plots TR against σ_v at five layers spanning the 18-block Gemma expert. Early layers ($\ell=1, 5$) sit near a fixed value ($\text{TR} \approx 4$ and ≈ 3.2 respectively) across the entire σ_v range indicating equal importance to all frames (1 current vs 4 future frames). For deep layers ($\ell=15, 18$): at low σ_v (clean futures) TR is high, and as σ_v rises (noisier futures) TR collapses toward 1 or below, because the action head reroutes its attention onto the clean current-frame anchor. The mid-layer $\ell=10$ shows the same pattern with a smaller range, so the routing decision sharpens with depth.

D Implementation and Hyperparameter Details

D.1 Video Backbone Adaptation

LoRA adaptation. We adapt the DiT backbone (App. A.1) with Low-Rank Adaptation [27] at ranks $r \in \{16, 32, 64\}$ and $\alpha = r$. LoRA is inserted into every attention and MLP projection in all

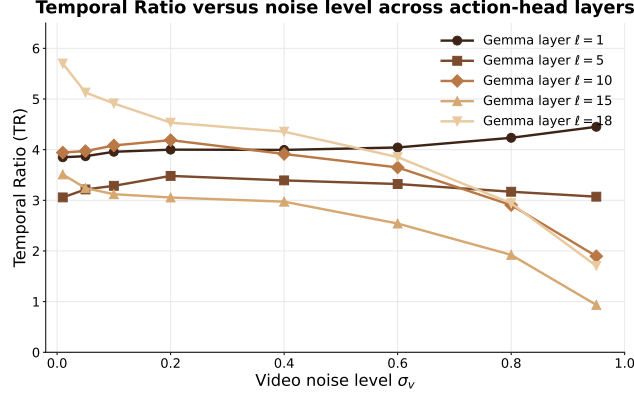


Figure 8: Temporal Ratio as a function of video noise level σ_v at five layers of the 18-layer Gemma action head (1-indexed). Shallow layers are insensitive to σ_v ; deep layers (and especially $\ell=15, 18$) carry the routing decision between the clean current frame and the noisy predicted futures. We pick $\ell^* = 15$ as the canonical TR layer for adaptive guidance and reported TR values.

28 blocks, which trains 2–6% of the backbone parameters ($\sim 40\text{M}$ at $r=16$, $\sim 180\text{M}$ at $r=64$); the rest of the DiT is frozen.

Full finetuning. We also evaluate updating all 2.1B DiT parameters. Optimizer settings match the LoRA configuration except for the learning rate, which is lowered from 10^{-4} to 10^{-5} to avoid destabilising the pretrained features.

Joint and two-stage training. Joint training (Alg. 1) runs for 50k steps with video loss weight $\lambda = 1.0$. Two-stage training runs Stage 1 (video only) for 25k steps and Stage 2 (action head only, video features cached) for 50k steps. Both modes share the optimizer, batch size, and noise schedule below.

Table 8: Video model finetuning hyperparameters.

Hyperparameter	LoRA	Full FT
Learning rate	1×10^{-4}	1×10^{-5}
LoRA rank r	32	–
LoRA α	$\alpha = r$	–
LoRA targets	all $Q/K/V/O$ and MLP projections, 28 blocks	–
Trainable params	2–6% of 2.1B	$\sim 2.1\text{B}$ (100%)
Optimizer	AdamW ($\beta_1=0.9, \beta_2=0.95, \epsilon=10^{-8}$)	same
Weight decay	1×10^{-8}	1×10^{-8}
Gradient clip (norm)	1.0	1.0
Training steps	50k joint, 25k Stage 1	50k joint
Batch size	8 per GPU $\times$ 16 GPUs = 128	same
LR schedule	cosine decay to 5×10^{-7}	same
EMA decay	0.999 (constant)	0.999
Video noise schedule	$\sigma_v \sim \mathcal{U}[0, 1]$	$\sigma_v \sim \mathcal{U}[0, 1]$
Joint loss weight λ	1.0	1.0

D.2 Action Head Training

Action flow matching. The action flow-matching loss and Euler-step inference procedure are defined in App. A.2. The empirical setting we use is $\sigma_a \sim \text{Beta}(1.5, 1.0) \cdot 0.999 + 0.001$ [26], which biases the training schedule toward high noise and matches the $\sigma_a=1$ initialisation at inference.

Action chunking. Action chunks are 17 steps and the policy replans every 10 environment steps (App. A.2). The action vector has 7 dimensions per arm on LIBERO (six joint deltas plus a binary gripper) and is padded to internal dimension 32, so the same network serves bimanual setups without architecture changes.

Table 9: Action head training hyperparameters.

Hyperparameter	Value
Architecture	Gemma expert (App. A.1; 18 layers, $\sim 300\text{M}$)
Action prediction	flow matching (single-step training, 10-step Euler inference)
Action chunk size	17 steps
Action dim	7 per arm (LIBERO); padded to internal dim 32
Proprioceptive dim	8, padded to internal dim 32
Action noise schedule	$\sigma_a \sim \text{Beta}(1.5, 1.0) \cdot 0.999 + 0.001$
Learning rate	1×10^{-4}
Training steps (joint and two-stage)	50k
Optimizer	AdamW ($\beta_1=0.9, \beta_2=0.95, \epsilon=10^{-8}$)
Weight decay	1×10^{-8}
Gradient clip (norm)	1.0

D.3 Feature Extraction

Extraction layer. We read intermediate features from block $k = 20$ of 28 in the DiT, matching the choice of [12]. The token shape, patchify scheme, and projection into the action head are described in App. A.1.

Video noise level. The 6-step FlowUniPC schedule (App. A.2) gives a grid of six σ_v values. We sweep the extraction index over $\{0, 1, 2, 4, 5\}$, omitting step 3 to bound the sweep size; this covers pure noise, the two near-noise levels, the mid-noise regime, and the near-clean regime. The resulting σ_v values are listed in Table ??.

Prediction horizon. Three temporal extents are evaluated at inference: $T \in \{5, 9, 24\}$ latent frames, corresponding to 17, 33, and 93 video frames at the $4\times$ temporal compression. Training is always done with a $T=5$ target. The horizon parameter only controls how many latent frames the video DiT rolls out during evaluation; the action head always receives the *first* 5 latent frames of the rollout regardless of T , so a larger T changes the quality of those first 5 frames (joint denoising over a longer chunk) rather than the number of frames the action head sees. $T = 5$ at inference matches training; $T = 24$ amounts to roughly 9 seconds of imagined future at 10Hz control. Plan guidance (App. D.4) selects T' from this same set.

Table 10: Feature extraction configurations.

Parameter	Values
Extraction layer k	20 (of 28 DiT blocks)
Latent temporal frames T	$\{5, 9, 24\}$ (= 17, 33, 93 video frames)
Spatial latent resolution $W \times H$	28×14 (from 448×224 input)
Feature dim D	2048
Video tokens per latent frame	98 (after $1 \times 2 \times 2$ patchify)
Video tokens fed to action head	$98T$
σ_v values (inf step $\rightarrow \sigma_v$)	0:1.000, 1:0.953, 2:0.883, 4:0.562, 5:0.024
Video solver steps	6 (FlowUniPC, shift 5.0)
Action Euler steps	10

674 D.4 Guidance Hyperparameters

675 **Language guidance.** We compute the null-text branch by replacing the text conditioning c
 676 with the zero embedding before the video forward pass. The resulting velocity difference
 677 $v_{\text{cond}} - v_{\text{uncond}}$ is scaled by $w_{\text{lang},i}$ at each action Euler step (Algorithm 3). The sweep covers
 678 $w_{\text{lang}}^{\text{base}} \in \{0.04, 0.06, 0.1, 0.2, 0.3, 0.5, 1.0\}$; the canonical setting in the main results is $w_{\text{lang}}^{\text{base}} = 0.1$.

679 **Plan guidance.** The plan branch queries the video model with the extended horizon $T' = 24$ latent
 680 frames (93 video frames at 10Hz, roughly 9 seconds of imagined future) and truncates the resulting
 681 features back to the policy horizon T before passing them to the action head. The velocity difference
 682 $v_{\text{plan}} - v_{\text{cond}}$ is scaled by $w_{\text{plan},i}$. The sweep covers $w_{\text{plan}}^{\text{base}} \in \{0.04, 0.06, 0.1, 0.2, 0.3, 0.5, 1.0\}$ with
 683 canonical $w_{\text{plan}}^{\text{base}} = 0.3$.

684 **TR-adaptive schedule.** Adaptive scaling reads TR_i from layer $\ell^* = 15$ of the action head’s atten-
 685 tion. The episode baseline TR_0 is set at the first replan of each episode and held until the episode
 686 terminates. The scale $s_i = \text{clip}(\text{TR}_i / \text{TR}_0 - 1, 0, s_{\text{max}})$ uses $s_{\text{max}} = 2.0$.

Table 11: Guidance hyperparameters.

Hyperparameter	Values
$w_{\text{lang}}^{\text{base}}$	0.1
$w_{\text{plan}}^{\text{base}}$	0.3
Planning horizon T'	$\{24\}$ latent frames
Adaptive cap s_{max}	2.0
TR extraction layer ℓ^*	15
Schedules	flat, TR-adaptive

687 E Real-World Setup and Evaluation Details

688 E.1 Hardware Description



Figure 9: **Bimanual real-world testbed.** Two I2RT YAM 6-DOF arms [47] and three Intel RealSense D405 [48]: one mounted top-down on the workspace and two more mounted on the wrists. The two arms together expose 12 joint DoF plus 2 gripper channels, which maps to the 14-dim action used by the policy (7 per arm). We use the RGB stream from three cameras where the native 1280×720 output is downsampled to 224×168 before being concatenated into the policy input.

689 E.2 Bimanual YAM Platform

690 **Observation space.** The policy reads three RGB cameras per timestep: a top-mounted workspace
691 view, a left-wrist camera, and a right-wrist camera. Each camera produces a 224×168 frame; the
692 three are concatenated horizontally into a single 672×168 image before input to the video backbone
693 (the same flat-image convention used for LIBERO; see App. B.1). Cameras run at 30Hz; the policy
694 applies a frame-skip of 3, so the 17 video frames it sees per replan span 51 source frames and form
695 a 10Hz 1.7-second window. After the Cosmos-Tokenizer $4\times$ temporal compression these 17 video
696 frames yield $T = 5$ latent frames, matching the simulation configuration. In addition we also use the
697 absolute joint angles for each inference step.

698 **Action space.** The action is 14-dim absolute joint angle targets, organised as 7 dimensions per
699 arm. The flow-matching head from simulation (App. D.2) is reused without architectural change;
700 only the action width and proprioceptive width differ. Internally the 14-dim action is zero-padded to
701 the action head’s internal width of 32. Each replan produces a chunk of 50 actions at 30Hz (a 1.67-
702 second open-loop horizon); the first 30 actions (1 second) are executed and the rest are discarded
703 before the next replan.

704 **Latent token count.** The 672×168 stacked image, after $8\times$ spatial compression by Cosmos-
705 Tokenizer-CV4x8x8, yields an 84×21 latent grid per frame. The DiT $1 \times 2 \times 2$ patchifies, pads and
706 groups spatial neighbours into $42 \times 11 = 462$ DiT tokens per latent frame, so the full $T = 5$ video
707 prefix presented to the action head contains $5 \times 462 = 2310$ tokens. For reference, the corresponding
708 LIBERO numbers are 98 tokens per latent frame and $5 \times 98 = 490$ total tokens at $T = 5$.

709 E.3 Real-World Training Dataset: PP24

710 **Dataset.** Real-world training uses a multi-task bimanual YAM dataset consisting of 24 tasks.
711 Overall, the dataset contains 5,600 episodes for a total of 9.98M raw camera frames at 30 fps.
712 (~ 3.3 M after frame-skipping at 10 fps). The natural-language instructions are paraphrased per
713 episode: the dataset has 1,036 unique prompt strings spread across the 24 task categories, so the
714 policy sees a wider distribution of instruction wordings.

715 **Task families.** The 24 tasks fall into four skill categories: *folding* (1 task), *garment manipulation*
716 (2 tasks; removing shirts and t-shirts from hangers), *high-precision assembly* (1 task), and *pick-and-*
717 *place* (20 tasks: bussing tables, loading and unloading dish racks, placing snacks and beverages on
718 shelves, throwing items into bins, handoffs, mug-orientation correction, and a relative-placement
719 task). Per-task episode counts are listed in Table 12.

720 **Sampling weights.** During training the action head sees a ConcatDataset of all 24 task reposi-
721 tories with a WeightedRandomSampler that assigns equal weight to each of the 24 tasks.

722 E.4 Real-World Compositional Evaluation

723 **Evaluation scenes.** All training scenarios had only one target receptacle and multiple instances of
724 the same target object. We construct three OOD scenes to evaluate compositional generalisation.
725 Each scene contains multiple target candidates and multiple receptacle candidates. **Scene 1** places a
726 paper bag and a bin together with fruits, testing whether the policy can bind the named target object
727 to the correct receptacle. **Scene 2** mixes snacks and bottles against the bag and bin, testing the same
728 binding under a multi-object multi-receptacle shift. **Scene 3** presents snacks together with a bin
729 and a shelf, testing whether the policy overfits to the *snack-on-shelf* behaviour seen frequently
730 during training.

731 **Task definitions and trials.** Each scene contains a fixed set of language instructions. The eight
732 evaluation tasks are listed in Table 13. Tasks 1 and 2 are evaluated for 15 trials each (5 trials with 3
733 fruits each); tasks 3 through 8 are evaluated for 5 trials each. Total budget per method: 60 trials. A

Table 12: Multi-task dataset task list, organised by skill family. Episode counts sum to 5,600.

Skill family	Task	Episodes
folding	Fold a pile of t-shirts and stack them	200
garment manipulation	Remove the shirt from the hanger	200
	Remove the t-shirt from the hanger	200
high-precision assembly	Unscrew bottle caps	200
pick-and-place	Place mixed dishes and glasses into a plastic bin	200
	Place plates into a plastic bin	200
	Bimanual object handoff	200
	Load bowls into a tabletop dish rack	200
	Load cups into a tabletop dish rack	200
	Load mixed dishes into a tabletop dish rack	200
	Load plates into a tabletop dish rack	200
	Place and organize beverages on a shelf	200
	Place and organize canned foods on a counter	200
	Place and organize chip bags on a shelf	200
	Place fake fruits into a fruit bowl	200
	Place snacks into a paper bag	200
	Place an object at a specified relative position	1000
	Take snacks out of a paper bag	200
	Throw plastic bottles into a bin	200
	Turn a mug right-side up	200
	Unload bowls from a tabletop dish rack	200
	Unload cups from a tabletop dish rack	200
	Unload mixed dishes from a tabletop dish rack	200
	Unload plates from a tabletop dish rack	200

trial succeeds when the robot approaches and picks up the named target object first and reaches the named receptacle.

Table 13: Real-world evaluation tasks, organised by scene.

Task	Instruction	Scene	Trials
1	Throw away the fruits into the bin	1	15
2	Place the fruits into the paper bag	1	15
3	Place snacks into the paper bag	2	5
4	Throw away the snacks into the bin	2	5
5	Place plastic bottle into the paper bag	2	5
6	Throw away the plastic bottle into the bin	2	5
7	Place snacks into the bin	3	5
8	Arrange snacks on the shelf	3	5
Total trials per method			60

E.5 Real-World Training Hyperparameters and Compute

Optimisation. We deploy joint LoRA training only in the real-world and use the same optimiser configuration as the simulation runs (App. D.1, App. D.2). The video DiT is adapted with LoRA at $r = 32$, $\alpha = 32$, learning rate 1×10^{-4} . The action head trains at peak learning rate 5×10^{-4} with 500 warmup steps and cosine decay to 5×10^{-5} over 10,000 steps. EMA decay is 0.999, weight decay 1×10^{-8} , gradient clip 1.0. Precision is bfloat16.

Baselines. We compare against π_0 [24], $\pi_{0.5}$ [26], and Cosmos-Policy [10], all retrained on the same dataset. All WAM and VAM baselines use the same Cosmos-Predict 2.5-2B video backbone as our model; the π -family baselines are at a comparable 3B PaliGemma VLM scale.

745 **Compute budget.** Each method is trained for 1M optimiser steps on an equivalent of 32 NVIDIA
 746 H200 80GB GPUs at matched effective batch size and bfloat16 precision.

Table 14: Real-world training compute and key hyperparameters.

Parameter	Value
GPU type	NVIDIA H200 80GB
Number of GPUs	32
Effective batch size	512
Training steps	1M
Precision	bfloat16
LoRA rank r / α	32 / 32
LoRA learning rate	1×10^{-4}
Action-head peak LR	5×10^{-4}
EMA decay	0.999

747 E.6 Inference Cost

748 **Forward-pass cost.** The unguided policy runs one video forward pass and one action-head 10-step
 749 Euler integration per replan. Language guidance adds a null-text video pass, plan guidance adds an
 750 extended-horizon video pass, and combined guidance adds both. The video forward-pass count per
 751 replan is therefore 1 for unguided, 2 for language-only or plan-only guidance, and 3 for combined
 752 guidance; the action-head cost is unchanged across all four configurations.

753 All videos are included in supplementary and <https://temporal-ratio.github.io/>.